skt aleph 대전A반

React와 Flask로 감시 대시보드 완성하기

아직 통과하지 못했습니다

설명

React와 Flask로 감시 대시보드 완성하기

과제 목표

지금까지 만든 `mini-watch`를 이어서 발전시킵니다. 일반 서비스의 게시판은 유지하고, 감시 서비스에 React 대시보드를 완성합니다.

운영자가 로그인한 뒤 일반 서비스의 요청 기록을 확인하고, 관찰한 내용을 메모로 남길 수 있어야 합니다. 이번 결과물은 다음 수업과 최종 프로젝트에서 계속 사용합니다.

수업 제공 코드와 DB 준비 자료를 사용해도 됩니다. 기능을 연결하고 화면 동작과 파일별 역할을 설명할 수 있도록 정리합니다. 화면 디자인은 자유입니다.

사용할 기술과 작업 범위

구성	사용할 기술과 역할
감시 화면	React로 화면을 구성하고 state와 props로 연결합니다.
프론트엔드 개발 환경	Vite로 개발 서버를 실행합니다.
감시 API	Flask가 JSON 요청을 받아 계정을 확인하고 데이터를 처리합니다.
데이터 저장	PostgreSQL에 계정·요청 기록·관찰 메모를 저장합니다.
DB 연결	Python의 psycopg와 기존 `connect_db()`를 사용합니다.

기존 `general`의 게시판과 요청 기록 전송 기능을 유지합니다. React 화면은 `monitor/frontend`, 감시 API는 `monitor/backend`에서 작업합니다. 프론트엔드가 DB에 직접 접속하지 않고 Flask API를 거쳐 데이터를 읽고 변경하도록 구성합니다.

로컬에서 일반 서비스는 `5100`, 감시 API는 `5200`, 감시 화면은 `5173`을 사용합니다. Vite의 `/api` 프록시로 React의 요청을 감시 Flask에 전달합니다.

1. 로그인 폼에서 대시보드로 이동하기

아이디와 비밀번호를 입력하면 React가 `POST /api/auth/login`으로 JSON을 보냅니다. Flask는 DB에서 사용자를 찾고, 비밀번호 해시 검증 함수로 입력한 비밀번호가 맞는지 확인합니다.

계정이 맞으면 반환된 사용자 정보를 React state에 저장하고 대시보드에 사용자 이름을 표시합니다. 계정이 틀리면 로그인 폼에 오류 안내를 표시합니다. 빈 입력은 `400`, 일치하지 않는 계정 정보는 `401`로 구분합니다.

로그아웃 버튼을 누르면 사용자 state를 비우고 로그인 폼으로 돌아갑니다. **본문 실습 기준으로 새로고침하면 로그인 폼으로 돌아오는 것이 정상입니다.** 이때 DB에 저장한 메모와 요청 기록은 유지되어야 합니다.

실제 서비스에서는 쿠키·세션으로 로그인 상태를 유지하고 서버에서도 접근을 검사합니다. 이번 필수 과제는 계정 판정과 화면 전환까지이며, 로그인 유지와 API 접근 제한은 아래 심화 과제에서 선택합니다.

2. 일반 서비스의 요청 기록 확인하기

대시보드에서 `GET /api/events`를 호출하여 실제 DB에 저장된 요청 기록을 표시합니다. 요청 방법, 경로, 응답 상태 코드를 확인할 수 있도록 구성합니다.

일반 게시판에 접속한 뒤 감시 화면에서 기록을 다시 조회하면 해당 요청이 보여야 합니다. 목록이 비어 있을 때와 조회에 실패했을 때는 상황에 맞는 안내를 표시합니다.

요청 기록은 일반 서비스가 자동으로 보내는 자료입니다. 운영자가 직접 작성·수정·삭제할 자료는 다음 절의 **관찰 메모**입니다.

3. 관찰 메모 작성·조회·수정·삭제하기

관찰 메모는 운영자가 남기는 제목과 내용입니다. 예를 들어 "없는 게시글 요청 확인"이라는 제목으로 "요청 경로와 404 응답을 확인했다."라고 작성합니다.

기능	API	화면에서 확인할 결과
목록 조회	`GET /api/notes`	저장된 메모의 번호와 제목을 표시합니다.
상세 조회	`GET /api/notes/<번호>`	선택한 메모의 제목과 내용을 표시합니다.
작성	`POST /api/notes`	새 메모를 저장하고 목록과 상세에 반영합니다.
수정	`PUT /api/notes/<번호>`	같은 번호의 메모를 수정하고 변경 내용을 표시합니다.
삭제	`DELETE /api/notes/<번호>`	확인한 메모를 삭제하고 목록과 상세 선택을 갱신합니다.

목록은 API 응답을 state에 저장한 뒤 `map()`으로 표시하며, 각 항목에는 메모 번호를 `key`로 사용합니다. 제목을 누르면 해당 번호로 상세를 조회합니다. 메모가 없으면 빈 목록 안내를 보여 줍니다.

작성 화면에서는 제목과 내용을 입력받습니다. 수정 화면에서는 기존 값을 먼저 보여 주며, 저장한 경우에만 DB를 변경합니다. 수정 중 취소하면 원래 내용이 유지되어야 합니다.

삭제는 확인 화면을 거칩니다. 확인 화면을 여는 것만으로 삭제하지 않으며, 취소하면 메모를 보존하고 삭제를 확정한 경우에만 API를 호출합니다.

저장·수정·삭제가 성공하면 목록을 다시 조회하여 DB 결과와 화면을 맞춥니다. 브라우저를 새로고침하고 다시 로그인해도 저장한 메모와 수정·삭제 결과가 유지되어야 합니다.

4. 잘못된 입력과 실패 응답 처리하기

Flask는 제목과 내용의 앞뒤 공백을 제거한 뒤 검사합니다. 어느 한쪽이 비어 있거나 공백뿐이면 `400`과 오류 안내를 반환하며, 새 메모를 저장하거나 기존 내용을 변경하지 않습니다.

존재하지 않는 번호의 메모를 조회·수정·삭제하면 `404`와 오류 안내를 반환합니다. 삭제한 번호를 다시 요청하는 경우에도 같은 기준을 적용합니다.

React는 서버의 성공·실패 응답을 확인하고 안내를 표시합니다. 실패한 요청을 저장 완료로 표시하지 않습니다.

5. 동작을 유지하며 모듈로 나누기

화면 컴포넌트는 `components`, API 요청 함수는 `api`에 나눕니다. `App.jsx`는 로그인 전후 화면을 연결하고, 부모와 자식 컴포넌트는 props로 자료와 이벤트 함수를 주고받습니다.

Flask는 기존 모듈 구조를 이어 사용합니다. `routes`는 HTTP 요청·응답, `repositories`는 SQL 실행, `db.py`는 DB 연결을 담당하도록 구성합니다. 입력값은 SQL의 `%s` 매개변수로 전달합니다.

아래는 이번 작업의 주요 구조입니다. `...`는 생략한 기존 파일입니다.

파일로 저장 복사

```
mini-watch-day04/
├── general/
│   └── ...                          # 기존 게시판과 기록 전송 유지
├── monitor/
│   ├── backend/
│   │   ├── app.py                   # 앱 생성·라우터 등록·실행
│   │   ├── routes/                  # 로그인·요청 기록·메모 API
│   │   ├── repositories/            # 계정·요청 기록·메모 SQL
│   │   ├── db.py
│   │   └── ...
│   └── frontend/
│       ├── src/
│       │   ├── App.jsx
│       │   ├── components/          # 로그인·대시보드·목록·상세·폼
│       │   ├── api/                 # 공통 요청·로그인·기록·메모
│       │   └── ...
│       └── ...
└── ...
```

파일을 분리한 뒤에도 로그인 판정, 요청 기록 조회, 메모 CRUD가 동일하게 동작해야 합니다.

통합 확인 시나리오

완성한 두 서비스를 함께 실행하고 다음 순서로 확인합니다.

1. 일반 게시판에서 실제 게시글을 조회하고, 존재하지 않는 게시글 주소에도 접속합니다.
2. 감시 화면에서 틀린 비밀번호로 실패 안내를 확인한 뒤, 올바른 계정으로 로그인합니다.
3. 요청 기록을 다시 조회하여 앞에서 접속한 경로와 응답 상태 코드를 비교합니다.
4. 관찰한 내용을 메모로 작성하고 목록·상세에 같은 내용이 나오는지 확인합니다.
5. 메모를 수정한 뒤, 공백뿐인 내용으로도 수정을 시도하여 오류 안내와 기존 내용 보존을 확인합니다.
6. 별도의 테스트 메모로 삭제 취소와 삭제 확정을 각각 확인합니다.
7. 새로고침하고 다시 로그인하여 남겨 둔 메모와 삭제 결과가 유지되는지 확인합니다.
8. 로그아웃한 뒤 로그인 폼으로 돌아오는지 확인합니다.

제출 안내

강의실에서 제공하는 GitHub 주소 또는 ZIP 제출 방식으로 실행 가능한 프로젝트를 제출합니다. 수업 제공 코드를 사용한 경우 README에 사용한 시작 자료와 직접 구현·수정한 부분을 적습니다.

제출물에는 다음 자료를 포함합니다.

- 일반 서비스와 감시 서비스의 소스 코드.
- Python의 `requirements.txt`와 프론트엔드의 `package.json`, `package-lock.json`.
- 필요한 DB·테이블·실습 계정 준비 방법과 SQL 파일. 수업 준비본을 그대로 사용했다면 해당 자료와 실행할 파일·순서를 README에 명시합니다.
- 실제 비밀번호를 제외한 `.env.example`과 각 설정값의 용도.

- Windows CMD 기준으로 설치·DB 준비·서비스 실행 순서, 접속 주소, 테스트용 계정 생성 방법을 적은 README.
- 구현한 필수 기능, 선택한 심화 기능, 통합 확인 결과를 적은 README.

실제 `.env`, 비밀번호·토큰, `venv` · `.venv`, `node_modules`, 캐시 파일은 제출에서 제외합니다. 다른 사람이 제출물을 받아 안내 순서대로 실행할 수 있는지 확인합니다.

심화 과제 — 선택

필수 기능을 완성한 뒤 다음 중 원하는 기능을 추가합니다. 심화 기능을 구현하지 않아도 필수 과제의 누락으로 평가하지 않습니다.

1. **요청 기록 검색·필터:** 경로 검색이나 응답 상태 코드 조건을 추가하고, 조건을 해제하면 전체 목록으로 돌아오게 합니다.
2. **요청 건수 요약:** 전체 요청 건수와 오류 요청 건수를 표시합니다. 오류의 기준과 집계 범위를 명시하고, 목록의 해당 자료와 숫자가 일치하는지 확인합니다.
3. **메모 처리 상태:** "확인 전", "확인 중", "완료" 같은 상태를 추가합니다. 상태를 변경하면 DB에 저장되고 다시 조회해도 유지되도록 합니다.
4. **세션 유지와 API 보호:** 교안 부록을 참고하여 새로고침 후에도 로그인을 유지합니다. 로그인하지 않은 상태에서 보호 대상 메모 API와 요청 기록 조회 API를 직접 호출하면 서버가 거절하는지도 확인합니다.

체크리스트 필수 20개를 전부 충족하면 통과

보통 채점 지침

요구의 뜻을 달성하면 충족합니다. 대소문자·문장부호·공백·줄바꿈 같은 사소한 표기 차이는 봐줍니다.

Flask 백엔드와 React 프론트엔드를 실행하고 API로 연결했다.	필수
PostgreSQL에 psycopg로 연결하고 SQL에 입력값을 전달할 때 매개변수 바인딩을 사용했다.	필수
작성·수정한 자료가 DB에 저장되며 프로그램을 다시 실행해도 유지된다.	필수
로그인 입력값을 React state로 관리하고 Flask API로 전송한다.	필수
Flask에서 DB의 계정과 비밀번호 해시를 확인해 로그인 성공·실패를 판정한다.	필수
로그인 성공 시 대시보드를 표시하고 로그아웃 시 로그인 화면으로 돌아간다.	필수
일반 서비스에서 수집한 요청 기록의 메서드·경로·상태 코드를 표시한다.	필수
관찰 메모 목록을 표시하고 선택한 메모의 상세 내용을 조회한다.	필수
새 메모를 작성하면 DB에 저장되고 목록에 반영된다.	필수
기존 메모의 내용을 불러와 수정하고 DB와 화면에 반영한다.	필수
삭제 확인·취소를 제공하고 확정한 메모만 DB와 화면에서 제거한다.	필수

목록 새로고침 버튼으로 최신 메모와 요청 기록을 다시 조회한다.	필수
빈 값이나 공백뿐인 작성·수정 요청을 서버에서 400으로 거절하고 기존 자료를 유지한다.	필수
존재하지 않는 메모의 조회·수정·삭제 요청에 404를 반환한다.	필수
로그인이나 API 요청에 실패하면 화면에 오류 메시지를 표시한다.	필수
React 화면을 역할별 컴포넌트로 분리하고 필요한 데이터와 함수를 props로 전달한다.	필수
React의 API 요청 함수를 화면 컴포넌트와 별도 파일로 분리한다.	필수
Flask의 라우터·DB 처리를 분리하고 app.py에서 연결한다.	필수
환경 설정 예시를 제공하고 실제 비밀번호가 담긴 .env는 Git에서 제외한다.	필수
패키지 설치 파일과 README를 제공해 DB 준비부터 서버·프론트 실행까지 따라 할 수 있게 한다.	필수
요청 기록을 경로나 상태 코드로 검색·필터링한다.	선택
조회한 기록의 전체 요청 수와 오류 요청 수를 표시한다.	선택
관찰 메모에 처리 상태를 추가하고 변경한 상태를 DB에 저장한다.	선택
쿠키·세션으로 로그인을 유지하고 보호할 API에서도 로그인 여부를 검사한다.	선택

제출

낼 형식을 고르세요

zip 파일

GitHub 저장소

내 시도

아직 낸 시도가 없습니다.

개인정보처리방침 이용약관